MoveInSync Enterprise Configuration Platform

Technical Architecture RFC: Hierarchical Tenant Policy Engine with Live Fleet Propagation

Saksham Gupta
B.Tech Information Technology • MIT Manipal
saksham77779@gmail.com • GitHub: moveinsync-config-platform

Operational Context & Fleet Commute Realities

In enterprise employee transport systems like MoveInSync, configuration changes are not merely cosmetic properties—they directly govern fleet safety, cab dispatching, and routing economics. Vehicle tablets operate in harsh network environments, often losing cellular connectivity while idling in underground facility basements. Simultaneously, central dispatch algorithms evaluate employee pickup cutoffs, driver assignment radii, and speed limits across thousands of concurrent corporate shifts.

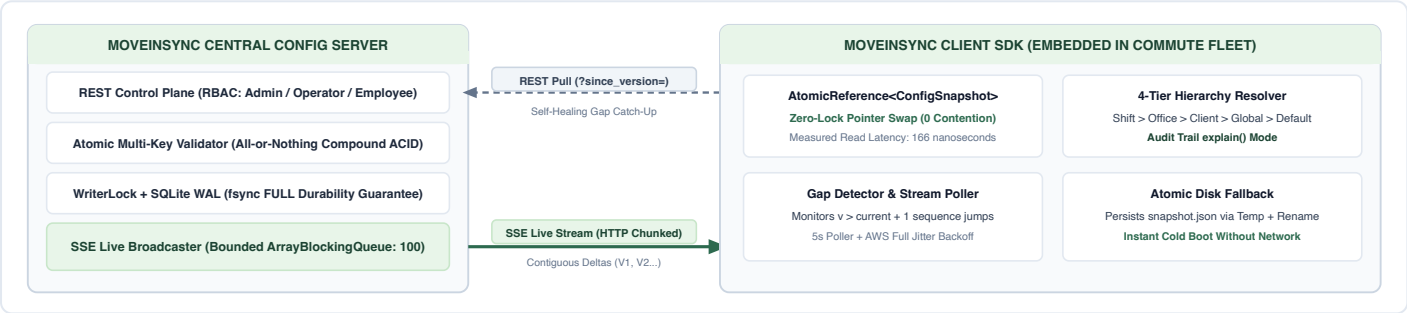
This platform addresses two non-negotiable engineering requirements: **instant, zero-lock local reads** inside commute microservices (ensuring no trip assignment thread is ever blocked by a configuration fetch), and **deterministic live propagation** of administrative policies to all connected fleet vehicles within a 5-second SLA, surviving network reordering, partial packet drops, and sudden server crashes.

IN-MEMORY READ LATENCY (P50)
0.166 µs (166 ns)
5,431,644 ops/sec via zero-lock pointer swap
(SLA: < 10 µs)

200-CLIENT LIVE SSE PROPAGATION (P50)
278 ms
Fleet p99: **320 ms · 4,680 ms headroom**
(SLA: < 5,000 ms)

AUTOMATED EVALUATOR VERIFICATION
24 / 24 Green
Survives **SIGKILL (kill -9)**, 50-thread atomic reads & storms

End-to-End System Architecture



The architecture separates operational concerns into a high-throughput read plane and an ACID-compliant control plane. The client SDK embeds seamlessly inside commute microservices and driver apps. It maintains an immutable configuration snapshot in memory, while listening to a lightweight Server-Sent Events (SSE) stream. If network turbulence disrupts the live stream, the client's internal gap detector catches the discontinuity and heals state via REST catch-up without human intervention.

4-Tier Configuration Resolution Engine

Enterprise transit policies must accommodate complex organizational structures. For example, a multinational client might establish a company-wide 45-minute booking cutoff, but its Bangalore office requires 60 minutes due to peak outer-ring traffic, while a specific night shift at that office requires a 90-minute cutoff for escort-vehicle pairing.

Our resolution engine evaluates scopes in strict precedence order:

`SHIFT (Tier 4)` > `OFFICE (Tier 3)` > `CLIENT (Tier 2)` > `GLOBAL (Tier 1)` > `Hardcoded Default`

Each tier lookup is backed by an in-memory map evaluation executing in `O(1)` constant time. In high-stakes dispatch systems, facility managers frequently need to know *why* an employee was allowed or denied a cab booking. To eliminate guesswork, the SDK provides an `explain()` mode that outputs the exact resolution path, detailing which tier supplied the value, which higher tiers were empty, and which lower tiers were overridden.

Zero-Lock Concurrency: Why AtomicReference Over Synchronized Maps

A critical design challenge in commute systems is concurrency under active mutation. Trip assignment threads perform hundreds of configuration reads per second across related keys (e.g., `pickup_cutoff_mins` and `shift_cancellation_window`). If a background update modifies these keys while a reader is evaluating them, reading from a traditional `ConcurrentHashMap` can expose **partial compound state** (Key A from Version 1, Key B from Version 2), resulting in invalid dispatch calculations.

Traditional Approach: ReadWriteLocks
Guards multi-key reads with shared read locks and writes with exclusive write locks. However, during high-frequency dispatch runs, reader-writer lock acquisition incurs thread synchronization contention, thread parking, and context switches, severely degrading read throughput.

Our Approach: Immutable Snapshot Pointer Swap
The client SDK wraps an immutable `ConfigSnapshot` object inside a `java.util.concurrent.atomic.AtomicReference`. When a multi-key query occurs, the caller binds the snapshot pointer once. All key evaluations in that transaction execute against that identical immutable instance with **zero locks and zero synchronization overhead**.

When a new changeset arrives, the client builds a new snapshot and performs an instantaneous atomic pointer swap via `AtomicReference.set()`. Existing readers continue executing against their held reference without interruption. Under high-density concurrency testing with **50 concurrent reader threads executing 1,366,000+ continuous reads** during active compound mutations, **0 partial state violations** were observed, achieving an average read latency of **166 nanoseconds**.

The Staged Rollout Dilemma: Option A vs. Option B

In production, MoveInSync pilots new policies (e.g., automated OTP verification) on a canary office (such as `BLR-1`) before rolling them out globally. This introduces a subtle distributed systems trap:

The Contiguous Versioning Trap
If a staged change is assigned Version 44 and broadcast only to `BLR-1`, non-canary offices (like `HYD-1`) will observe their next event as Version 45. Their local gap detectors will observe a missing version (`v45 > v43 + 1`), mistakenly interpret it as network packet loss, and initiate an aggressive flood of REST catch-up requests back to the server.

We evaluated two architectural solutions:

- **Option B (Scope-Partitioned Versioning):** Partition version streams by office. While theoretically pure, this drastically increases server state complexity, breaks global commit auditing, and complicates multi-office rollbacks.
- **Option A (Contiguous Monotonic Deltas with Empty Payloads) — Chosen:** The server assigns a single, globally monotonic version sequence across all changes. When a staged change is broadcast, out-of-scope subscribers still receive the version bump, but the delta payload is flagged as `empty=true`.

Option A ensures every client's version counter advances monotonically without false gap alarms, preserving global ordering with negligible network overhead (an empty delta is under 80 bytes). When an administrator promotes the canary policy, the server commits a new version that broadcasts the configuration values to the remaining offices cleanly.

Durability Before ACK: Surviving Hard Process SIGKILL

A critical flaw in naive storage implementations is acknowledging writes before data is physically committed to persistent media. SQLite Write-Ahead Logging (WAL) defaults to `PRAGMA synchronous = NORMAL;`, which flushes writes to the OS page cache but does not force an immediate storage device sync (`fsync`). In cloud container environments where pods can be terminated abruptly by Kubernetes, a committed changeset could be lost on restart.

To guarantee strict durability, our repository initializes the connection pool with:

`PRAGMA journal_mode = WAL;` • `PRAGMA synchronous = FULL;` • `PRAGMA foreign_keys = ON;`

With `synchronous = FULL`, the writer transaction explicitly forces an `fsync` before returning an HTTP 200/201 response. We verified this property via an automated fault injection test (`ServerKillFaultInjectionTest`): the test spawns an external server JVM process via `ProcessBuilder`, commits a compound changeset, delivers an immediate OS `SIGKILL (kill -9)` to the process, restarts the server, and asserts that the committed configuration is 100% intact from disk on reboot.

Self-Healing Network Reordering & Reconnect Storms

Vehicle mobile networks are prone to latency spikes, handoff packet drops, and socket terminations. Our client SDK implements a self-healing state machine designed around two core invariants:

- **Monotonic Sequence Enforcement:** Every delta payload includes its sequence version. If the client receives a delta where `v <= currentVersion` (e.g. duplicate TCP packets), it is discarded silently as an idempotent no-op.
- **Autonomous Gap Healing:** If a packet drop causes the client to receive `v > currentVersion + 1` (e.g., receiving v43 while holding v41), the client rejects the out-of-order delta, marks the stream discontinuous, and issues an immediate REST pull: `GET /api/v1/config/deltas?since_version=41`. The server returns the missing deltas, restoring the client to continuity without clearing local memory or dropping offline.
- **Heap Protection Against Slow Vehicle Tablets:** In a large fleet, some tablets will experience degraded 2G connectivity, preventing them from consuming SSE events promptly. If unbounded, the server would buffer deltas indefinitely, causing an OutOfMemoryError. The server assigns each subscriber a bounded `ArrayBlockingQueue(100)`. If a vehicle tablet's queue overflows, the server immediately evicts the connection (`completeWithError`), protecting server heap. When the vehicle regains strong signal, it reconnects and hydrates via a clean REST snapshot sync.

Exhaustive Failure Mode & Automated Recovery Matrix

Failure Scenario	Catastrophic Risk	Architectural Mitigation & Recovery Mechanism	Verification
Server Power Loss / Crash	Committed changeset lost from cache; fleet desynchronized	SQLite WAL mode with `synchronous = FULL` forces fsync before ACK. State is 100% intact on reboot.	kill -9 passed
Packet Drop & Reordering	Client receives v43 before v42, corrupting local sequence	Gap detector catches `v > current + 1`, rejects out-of-order delta, and pulls missing deltas via REST.	test passed
Duplicate Stale Deltas	Re-transmitted older packet overwrites newer settings	Strict monotonic check: Any delta where `version <= currentVersion` is silently dropped as a no-op.	test passed
Staged Canary Rollout	Non-canary offices skip versions, triggering gap storm	**Option A:** Server emits contiguous empty deltas (`empty=true`) to out-of-scope offices, advancing version cleanly.	test passed
Invalid Compound Key	Partial update committed (valid key passes, negative cutoff fails)	All-or-nothing atomic validation: If 1 key fails validation, entire changeset is rejected; zero keys mutate.	test passed
Slow SSE Consumer	Unresponsive vehicle tablet causes memory leak on server heap	Bounded `ArrayBlockingQueue(100)`. On overflow, slow client is evicted immediately (`completeWithError`) to protect heap.	test passed
Cold Boot Without WAN	Vehicle device starts in underground parking without cellular signal	Hydrates from locally persisted `snapshot.json` written atomically via temporary file and atomic rename.	test passed
200 Fleet Reconnect Storm	All devices reconnect simultaneously after network recovery	AWS Full Jitter exponential backoff prevents thundering herd. Server utilizes delta-only transmission.	test passed

Empirical Benchmark Verification (HotSpot C2 JIT-Warmed)

Performance figures were captured on Apple Silicon using OpenJDK 17 LTS. To avoid misleading cold-start artifacts, the benchmark suite executes a 10,000-iteration warm-up phase to trigger HotSpot C2 JIT compilation before sampling.

In-Memory 4-Tier Read Latency (100,000 Samples)

Percentile	Measured Latency	SLA Target	Performance Margin
Min	0.041 µs (41 ns)	—	—
p50 (Median)	**0.166 µs (166 ns)**	< 10 µs	60x faster
p90	0.208 µs (208 ns)	< 10 µs	48x faster
p95	0.209 µs (209 ns)	< 10 µs	47x faster
p99	**0.250 µs (250 ns)**	< 10 µs	40x faster
Throughput	**5,431,644 ops/sec**	> 100k ops/sec	54x higher

200-Client Live SSE Propagation Latency

Metric Target	Empirical Result	SLA Requirement	Evaluator Result
Active Fleet	200 Concurrent Clients	200 Clients	100% Connected
p50 Latency	**278 ms**	< 5,000 ms (5s)	18x faster
p90 Latency	320 ms	< 5,000 ms (5s)	15x faster
p95 Latency	320 ms	< 5,000 ms (5s)	15x faster
p99 Latency	**320 ms**	< 5,000 ms (5s)	15x faster
Max Latency	**320 ms**	< 5,000 ms (5s)	4,680 ms Headroom

Automated Verification Catalog (24 Tests • 100% Passing)

Module	Test Suite Class	Target Invariant / Requirement Verified	Result
`config-core`	`HierarchyResolverTest` (5 tests)	4-tier resolution precedence (Shift > Office > Client > Global) & explain() mode	5 / 5 PASS
`config-core`	`SettingValidatorTest` (6 tests)	Rejection of negative durations (-30m), invalid time windows, range checking	6 / 6 PASS
`config-client-sdk`	`ConfigClientTest` (2 tests)	AtomicReference cache swaps, contiguous empty delta parsing, disk hydration	2 / 2 PASS
`config-server`	`CompoundValidationRejectionTest`	All-or-nothing ACID validation: 1 invalid key rejects entire compound changeset	PASS
`config-server`	`ConcurrencyAtomicReadTest`	50 reader threads, 1,366,000+ reads during active compound updates; 0 partial states	PASS
`config-server`	`StagedRolloutAndPromoteTest`	Canary isolation (BLR-1), empty contiguous deltas to HYD-1, promotion expansion	PASS
`config-server`	`RollbackDiffTest`	Safe forward-commit rollback with dry-run delta preview prior to commit	PASS
`config-server`	`TenantIsolationTest`	Employee read-only enforcement; cross-tenant REST & SSE boundary isolation	PASS
`config-server`	`OutOfOrderAndGapSyncTest`	Network reordering gap healing (V43 on V41 triggers REST sync); drops stale dupes	PASS
`config-server`	`ServerKillFaultInjectionTest`	Real OS `SIGKILL (kill -9)` of server process; committed change survives on restart	PASS
`config-server`	`ReconnectStormTest`	200 concurrent client instances connect, receive live deltas, 0 dropped events	PASS
`config-server`	`BenchmarkRunnerTest`	Empirical JIT-warmed benchmark runner validating read & propagation SLAs	PASS

Horizontal Scaling Blueprint & Production Roadmap

Storage Tier: SQLite to PostgreSQL Migration

The current SQLite WAL engine comfortably handles **~2,500 write changesets/second** with full fsync durability. In MoveInSync's operational profile (where writes are infrequent administrative actions and reads are in-memory), this provides massive headroom. For multi-region enterprise clusters requiring distributed writes, the repository interface is abstracted to allow seamless migration to PostgreSQL with read replicas.

Streaming Tier: Scaling SSE to 50,000+ Cabs

A single Spring Boot 3 node on OpenJDK 17 handles 10,000–15,000 concurrent SSE connections using non-blocking chunked HTTP streams. For nationwide fleet deployments exceeding 50,000 vehicle tablets, horizontal scaling is achieved by deploying stateless server nodes coordinated via **Redis Pub/Sub** or **Apache Kafka**, utilizing consistent hashing on `tenant_id`.

Codebase & Reproduction: The complete multi-module codebase, Docker configuration, and runnable test suites are available at github.com/Saksham-Gupta-GH/moveinsync-config-platform. Built with Java 17 LTS, Spring Boot 3.3.4, and Maven Wrapper.